

```
6    je    .L2
7    shrq  $2, %rdi
8    call  rfun
9    addq  %rbx, %rax
10   .L2:
11    popq  %rbx
12    ret
```

- A. rfun 存储在被调用者保存寄存器%rbx 中的值是什么？
- B. 填写上述 C 代码中缺失的表达式。

3.8 数组分配和访问

C 语言中的数组是一种将标量数据聚集成更大数据类型的方式。C 语言实现数组的方式非常简单，因此很容易翻译成机器代码。C 语言的一个不同寻常的特点是可以产生指向数组中元素的指针，并对这些指针进行运算。在机器代码中，这些指针会被翻译成地址计算。

优化编译器非常善于简化数组索引所使用的地址计算。不过这使得 C 代码和它到机器代码的翻译之间的对应关系有些难以理解。

3.8.1 基本原则

对于数据类型 T 和整型常数 N ，声明如下：
 $T\ A[N];$

起始位置表示为 x_A 。这个声明有两个效果。首先，它在内存中分配一个 $L \cdot N$ 字节的连续区域，这里 L 是数据类型 T 的大小(单位为字节)。其次，它引入了标识符 A ，可以用 A 来作为指向数组开头的指针，这个指针的值就是 x_A 。可以用 $0 \sim N-1$ 的整数索引来访问该数组元素。数组元素 i 会被存放在地址为 $x_A + L \cdot i$ 的地方。

作为示例，让我们来看看下面这样的声明：

```
char    A[12];
char    *B[8];
int      C[6];
double  *D[5];
```

这些声明会产生带下列参数的数组：

数组	元素大小	总的大小	起始地址	元素 i
A	1	12	x_A	$x_A + i$
B	8	64	x_B	$x_B + 8i$
C	4	24	x_C	$x_C + 4i$
D	8	40	x_D	$x_D + 8i$

数组 A 由 12 个单字节(char)元素组成。数组 C 由 6 个整数组成，每个需要 8 个字节。B 和 D 都是指针数组，因此每个数组元素都是 8 个字节。

x86-64 的内存引用指令可以用来简化数组访问。例如，假设 E 是一个 int 型的数组，而我们想计算 $E[i]$ ，在此，E 的地址存放在寄存器%rdx 中，而 i 存放在寄存器%rcx 中。然后，指令

```
movl (%rdx,%rcx,4),%eax
```

会执行地址计算 $x_E + 4i$ ，读这个内存位置的值，并将结果存放到寄存器%eax 中。允许的

伸缩因子 1、2、4 和 8 覆盖了所有基本简单数据类型的大小。

 **练习题 3.36** 考虑下面的声明：

```
short   S[7];
short   *T[3];
short   **U[6];
int      V[8];
double  *W[4];
```

填写下表，描述每个数组的元素大小、整个数组的大小以及元素 i 的地址：

数组	元素大小	整个数组的大小	起始地址	元素 i
S			x_S	
T			x_T	
U			x_U	
V			x_V	
W			x_W	

3.8.2 指针运算

C 语言允许对指针进行运算，而计算出来的值会根据该指针引用的数据类型的大小进行伸缩。也就是说，如果 p 是一个指向类型为 T 的数据的指针， p 的值为 x_p ，那么表达式 $p+i$ 的值为 $x_p + L \cdot i$ ，这里 L 是数据类型 T 的大小。

单操作数操作符 ‘&’ 和 ‘*’ 可以产生指针和间接引用指针。也就是，对于一个表示某个对象的表达式 $Expr$ ， $\&Expr$ 是给出该对象地址的一个指针。对于一个表示地址的表达式 $AExpr$ ， $*AExpr$ 给出该地址处的值。因此，表达式 $Expr$ 与 $*\&Expr$ 是等价的。可以对数组和指针应用数组下标操作。数组引用 $A[i]$ 等同于表达式 $*(A + i)$ 。它计算第 i 个数组元素的地址，然后访问这个内存位置。

扩展一下前面的例子，假设整型数组 E 的起始地址和整数索引 i 分别存放在寄存器 $\%rdx$ 和 $\%rcx$ 中。下面是一些与 E 有关的表达式。我们还给出了每个表达式的汇编代码实现，结果存放在寄存器 $\%eax$ (如果是数据) 或寄存器 $\%rax$ (如果是指针) 中。

表达式	类型	值	汇编代码
E	int^*	x_E	<code>movq %rdx,%rax</code>
$E[0]$	int	$M[x_E]$	<code>movl (%rdx),%rax</code>
$E[i]$	int	$M[x_E + 4i]$	<code>movl (%rdx,%rcx,4),%eax</code>
$\&E[2]$	int^*	$x_E + 8$	<code>leaq 8(%rdx),%rax</code>
$E+i-1$	int^*	$x_E + 4i - 4$	<code>leaq -4(%rdx,%rcx,4),%rax</code>
$*(E+i-3)$	int	$M[x_E + 4i - 12]$	<code>movl -12(%rdx,%rcx,4),%eax</code>
$\&E[i]-E$	long	i	<code>movq %rcx,%rax</code>

在这些例子中，可以看到返回数组值的操作类型为 int ，因此涉及 4 字节操作 (例如 `movl`) 和寄存器 (例如 $\%eax$)。那些返回指针的操作类型为 int^* ，因此涉及 8 字节操作 (例如 `leaq`) 和寄存器 (例如 $\%rax$)。最后一个例子表明可以计算同一个数据结构中的两个指针之差，结果的数据类型为 long ，值等于两个地址之差除以该数据类型的大小。

 **练习题 3.37** 假设短整型数组 S 的地址 x_S 和整数索引 i 分别存放在寄存器 $\%rdx$ 和 $\%rcx$ 中。对下面每个表达式，给出它的类型、值的表达式和汇编代码实现。如果结果

是指针的话，要保存在寄存器%rax 中，如果数据类型为 short，就保存在寄存器元素%ax 中。

表达式	类型	值	汇编代码
S+ 1			
S[3]			
&S[i]			
S[4*i+ 1]			
S+ i-5			

3.8.3 嵌套的数组

当我们创建数组的数组时，数组分配和引用的一般原则也是成立的。例如，声明 `int A[5][3];`

等价于下面的声明

```
typedef int row3_t[3];
row3_t A[5];
```

数据类型 `row3_t` 被定义为一个 3 个整数的数组。数组 `A` 包含 5 个这样的元素，每个元素需要 12 个字节来存储 3 个整数。整个数组的大小就是 $4 \times 5 \times 3 = 60$ 字节。

数组 `A` 还可以被看成一个 5 行 3 列的二维数组，用 `A[0][0]` 到 `A[4][2]` 来引用。数组元素在内存中按照“行优先”的顺序排列，意味着第 0 行的所有元素，可以写作 `A[0]`，后面跟着第 1 行的所有元素(`A[1]`)，以此类推，如图 3-36 所示。

这种排列顺序是嵌套声明的结果。将 `A` 看作一个有 5 个元素的数组，每个元素都是 3 个 `int` 的数组，首先是 `A[0]`，然后是 `A[1]`，以此类推。

要访问多维数组的元素，编译器会以数组起始为基地址，(可能需要经过伸缩的)偏移量为索引，产生计算期望的元素的偏移量，然后使用某种 `MOV` 指令。通常来说，对于一个声明如下的数组：

```
T D[R][C];
```

它的数组元素 `D[i][j]` 的内存地址为

$$\&D[i][j] = x_D + L(C \cdot i + j) \tag{3.1}$$

这里， L 是数据类型 T 以字节为单位的大小。作为一个示例，考虑前面定义的 5×3 的整型数组 `A`。假设 x_A 、 i 和 j 分别在寄存器%rdi,%rsi 和%rdx 中。然后，可以用下面的代码将数组元素 `A[i][j]` 复制到寄存器%eax 中：

```
A in %rdi, i in %rsi, and j in %rdx
1   leaq    (%rsi,%rsi,2), %rax    Compute 3i
2   leaq    (%rdi,%rax,4), %rax    Compute x_A + 12i
3   movl    (%rax,%rdx,4), %eax    Read from M[x_A + 12i + 4j]
```

行	元素	地址
A[0]	A[0][0]	x_A
	A[0][1]	$x_A + 4$
	A[0][2]	$x_A + 8$
A[1]	A[1][0]	$x_A + 12$
	A[1][1]	$x_A + 16$
	A[1][2]	$x_A + 20$
A[2]	A[2][0]	$x_A + 24$
	A[2][1]	$x_A + 28$
	A[2][2]	$x_A + 32$
A[3]	A[3][0]	$x_A + 36$
	A[3][1]	$x_A + 40$
	A[3][2]	$x_A + 44$
A[4]	A[4][0]	$x_A + 48$
	A[4][1]	$x_A + 52$
	A[4][2]	$x_A + 56$

图 3-36 按照行优先顺序存储的数组元素

正如可以看到的那样，这段代码计算元素的地址为 $x_A + 12i + 4j = x_A + 4(3i + j)$ ，使用了 x86-64 地址运算的伸缩和加法特性。

 **练习题 3.38** 考虑下面的源代码，其中 M 和 N 是用 `# define` 声明的常数：

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

在编译这个程序中，GCC 产生如下汇编代码：

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1  sum_element:
2      leaq    0(,%rdi,8), %rdx
3      subq    %rdi, %rdx
4      addq    %rsi, %rdx
5      leaq    (%rsi,%rsi,4), %rax
6      addq    %rax, %rdi
7      movq    Q(,%rdi,8), %rax
8      addq    P(,%rdx,8), %rax
9      ret
```

运用逆向工程技能，根据这段汇编代码，确定 M 和 N 的值。

3.8.4 定长数组

C 语言编译器能够优化定长多维数组上的操作代码。这里我们展示优化等级设置为 -O1 时 GCC 采用的一些优化。假设我们用如下方式将数据类型 `fix_matrix` 声明为 16×16 的整型数组：

```
#define N 16
typedef int fix_matrix[N][N];
```

(这个例子说明了一个很好的编码习惯。当程序要用一个常数作为数组的维度或者缓冲区的大小时，最好通过 `# define` 声明将这个常数与一个名字联系起来，然后在后面一直使用这个名字代替常数的数值。这样一来，如果需要修改这个值，只用简单地修改这个 `# define` 声明就可以了。)图 3-37a 中的代码计算矩阵 A 和 B 乘积的元素 i, k ，即 A 的行 i 和 B 的列 k 的内积。GCC 产生的代码(我们再反汇编成 C)，如图 3-37b 中函数 `fix_prod_ele_opt` 所示。这段代码包含很多聪明的优化。它去掉了整数索引 j ，并把所有的数组引用都转换成了指针间接引用，其中包括(1)生成一个指针，命名为 `Aptr`，指向 A 的行 i 中连续的元素；(2)生成一个指针，命名为 `Bptr`，指向 B 的列 k 中连续的元素；(3)生成一个指针，命名为 `Bend`，当需要终止该循环时，它会等于 `Bptr` 的值。`Aptr` 的初始值是 A 的行 i 的第一个元素的地址，由 C 表达式 `&A[i][0]` 给出。`Bptr` 的初始值是 B 的列 k 的第一个元素的地址，由 C 表达式 `&B[0][k]` 给出。`Bend` 的值是假想中 B 的列 j 的第 $(n+1)$ 个元素的地址，由 C 表达式 `&B[N][k]` 给出。

下面给出的是 GCC 为函数 `fix_prod_ele` 生成的这个循环的实际汇编代码。我们看到 4 个寄存器的使用如下：`%eax` 保存 `result`，`%rdi` 保存 `Aptr`，`%rcx` 保存 `Bptr`，而 `%rsi` 保存 `Bend`。

```

/* Compute i,k of fixed matrix product */
int fix_prod_ele (fix_matrix A, fix_matrix B, long i, long k) {
    long j;
    int result = 0;

    for (j = 0; j < N; j++)
        result += A[i][j] * B[j][k];

    return result;
}

```

a) 原始的C代码

```

1  /* Compute i,k of fixed matrix product */
2  int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k) {
3      int *Aptr = &A[i][0];    /* Points to elements in row i of A */
4      int *Bptr = &B[0][k];    /* Points to elements in column k of B */
5      int *Bend = &B[N][k];    /* Marks stopping point for Bptr */
6      int result = 0;
7      do {
8          result += *Aptr * *Bptr; /* Add next product to sum */
9          Aptr++;                /* Move Aptr to next column */
10         Bptr += N;             /* Move Bptr to next row */
11     } while (Bptr != Bend);    /* Test for stopping point */
12     return result;
13 }

```

b) 优化过的C代码

图 3-37 原始的和优化过的代码，该代码计算定长数组的矩阵乘积的元素 i , k 。
编译器会自动完成这些优化

```
int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k)
```

```
A in %rdi, B in %rsi, i in %rdx, k in %rcx
```

```

1  fix_prod_ele:
2      salq    $6, %rdx                Compute 64 * i
3      addq    %rdx, %rdi              Compute Aptr =  $x_A + 64i = \&A[i][0]$ 
4      leaq    (%rsi,%rcx,4), %rcx     Compute Bptr =  $x_B + 4k = \&B[0][k]$ 
5      leaq    1024(%rcx), %rsi        Compute Bend =  $x_B + 4k + 1024 = \&B[N][k]$ 
6      movl    $0, %eax               Set result = 0
7  .L7:                                loop:
8      movl    (%rdi), %edx            Read *Aptr
9      imull   (%rcx), %edx            Multiply by *Bptr
10     addl    %edx, %eax              Add to result
11     addq    $4, %rdi                Increment Aptr ++
12     addq    $64, %rcx               Increment Bptr += N
13     cmpq    %rsi, %rcx              Compare Bptr:Bend
14     jne     .L7                     If !=, goto loop
15     rep; ret                        Return

```



练习题 3.39 利用等式 3.1 来解释图 3-37b 的 C 代码中 Aptr、Bptr 和 Bend 的初始值计算(第 3~5 行)是如何正确反映 fix_prod_ele 的汇编代码中它们的计算(第 3~5 行)的。

 **练习题 3.40** 下面的 C 代码将定长数组的对角线上的元素设置为 val:

```
/* Set all diagonal elements to val */
void fix_set_diag(fix_matrix A, int val) {
    long i;
    for (i = 0; i < N; i++)
        A[i][i] = val;
}
```

当以优化等级 -O1 编译时, GCC 产生如下汇编代码:

```
1  fix_set_diag:
   void fix_set_diag(fix_matrix A, int val)
   A in %rdi, val in %rsi
2      movl    $0, %eax
3      .L13:
4      movl    %esi, (%rdi,%rax)
5      addq    $68, %rax
6      cmpq    $1088, %rax
7      jne     .L13
8      rep; ret
```

创建一个 C 代码程序 `fix_set_diag_opt`, 它使用类似于这段汇编代码中所使用的优化, 风格与图 3-37b 中的代码一致。使用含有参数 N 的表达式, 而不是整数常量, 使得如果重新定义了 N , 你的代码仍能够正确地工作。

3.8.5 变长数组

历史上, C 语言只支持大小在编译时就能确定的多维数组(对第一维可能有些例外)。程序员需要变长数组时不得不用 `malloc` 或 `calloc` 这样的函数为这些数组分配存储空间, 而且不得不显式地编码, 用行优先索引将多维数组映射到一维数组, 如公式(3.1)所示。ISO C99 引入了一种功能, 允许数组的维度是表达式, 在数组被分配的时候才计算出来。

在变长数组的 C 版本中, 我们可以将一个数组声明如下:

```
int A[expr1][expr2]
```

它可以作为一个局部变量, 也可以作为一个函数的参数, 然后在遇到这个声明的时候, 通过对表达式 `expr1` 和 `expr2` 求值来确定数组的维度。因此, 例如要访问 $n \times n$ 数组的元素 i, j , 我们可以写一个如下的函数:

```
int var_ele(long n, int A[n][n], long i, long j) {
    return A[i][j];
}
```

参数 n 必须在参数 `A[n][n]` 之前, 这样函数就可以在遇到这个数组的时候计算出数组的维度。

GCC 为这个引用函数产生的代码如下所示:

```
int var_ele(long n, int A[n][n], long i, long j)
n in %rdi, A in %rsi, i in %rdx, j in %rcx
1  var_ele:
2      imulq    %rdx, %rdi                Compute  $n \cdot i$ 
3      leaq     (%rsi,%rdi,4), %rax        Compute  $x_A + 4(n \cdot i)$ 
4      movl     (%rax,%rcx,4), %eax        Read from  $M[x_A + 4(n \cdot i) + 4j]$ 
5      ret
```

正如注释所示，这段代码计算元素 i, j 的地址为 $x_A + 4(n \cdot i) + 4j = x_A + 4(n \cdot i + j)$ 。这个地址的计算类似于定长数组的地址计算(参见 3.8.3 节)，不同点在于 1) 由于增加了参数 n ，寄存器的使用变化了；2) 用了乘法指令来计算 $n \cdot i$ (第 2 行)，而不是用 `leaq` 指令来计算 $3i$ 。因此引用变长数组只需要对定长数组做一点儿概括。动态的版本必须用乘法指令对 i 伸缩 n 倍，而不能用一系列的移位和加法。在一些处理器中，乘法会招致严重的性能处罚，但是在这种情况下无可避免。

在一个循环中引用变长数组时，编译器常常可以利用访问模式的规律性来优化索引的计算。例如，图 3-38a 给出的 C 代码，它计算两个 $n \times n$ 矩阵 A 和 B 乘积的元素 i, k 。GCC 产生的汇编代码，我们再重新变为 C 代码(图 3-38b)。这个代码与固定大小数组的优化代码(图 3-37)风格不同，不过这更多的是编译器选择的结果，而不是两个函数有什么根本的不同造成的。图 3-38b 的代码保留了循环变量 j ，用以判定循环是否结束和作为到 A 的行 i 的元素组成的数组的索引。

```

1  /* Compute i,k of variable matrix product */
2  int var_prod_ele(long n, int A[n][n], int B[n][n], long i, long k) {
3      long j;
4      int result = 0;
5
6      for (j = 0; j < n; j++)
7          result += A[i][j] * B[j][k];
8
9      return result;
10 }
```

a) 原始的C代码

```

/* Compute i,k of variable matrix product */
int var_prod_ele_opt(long n, int A[n][n], int B[n][n], long i, long k) {
    int *Arow = A[i];
    int *Bptr = &B[0][k];
    int result = 0;
    long j;
    for (j = 0; j < n; j++) {
        result += Arow[j] * *Bptr;
        Bptr += n;
    }
    return result;
}
```

b) 优化后的C代码

图 3-38 计算变长数组的矩阵乘积的元素 i, k 的原始代码和优化后的代码。编译器自动执行这些优化

下面是 `var_prod_ele` 的循环的汇编代码：

```

Registers: n in %rdi, Arow in %rsi, Bptr in %rcx
          4n in %r9, result in %eax, j in %edx
1  .L24:                                loop:
2      movl    (%rsi,%rdx,4), %r8d      Read Arow[j]
3      imull   (%rcx), %r8d             Multiply by *Bptr
```

4	addl	%r8d, %eax	Add to result
5	addq	\$1, %rdx	j++
6	addq	%r9, %rcx	Bptr += n
7	cmpq	%rdi, %rdx	Compare j:n
8	jne	.L24	If !=, goto loop

我们看到程序既使用了伸缩过的值 $4n$ (寄存器 `%r9`) 来增加 `Bptr`, 也使用了 n 的值 (寄存器 `%rdi`) 来检查循环的边界。C 代码中并没有体现出需要这两个值, 但是由于指针运算的伸缩, 才使用了这两个值。

可以看到, 如果允许使用优化, GCC 能够识别出程序访问多维数组的元素的步长。然后生成的代码会避免直接应用等式 (3.1) 会导致的乘法。不论生成基于指针的代码 (图 3-37b) 还是基于数组的代码 (图 3-38b), 这些优化都能显著提高程序的性能。

3.9 异质的数据结构

C 语言提供了两种将不同类型的对象组合到一起创建数据类型的机制: 结构 (structure), 用关键字 `struct` 来声明, 将多个对象集合到一个单位中; 联合 (union), 用关键字 `union` 来声明, 允许用几种不同的类型来引用一个对象。

3.9.1 结构

C 语言的 `struct` 声明创建一个数据类型, 将可能不同类型的对象聚合到一个对象中。用名字来引用结构的各个组成部分。类似于数组的实现, 结构的所有组成部分都存放在内存中一段连续的区域内, 而指向结构的指针就是结构第一个字节的地址。编译器维护关于每个结构类型的信息, 指示每个字段 (field) 的字节偏移。它以这些偏移作为内存引用指令中的位移, 从而产生对结构元素的引用。

给 C 语言初学者 将一个对象表示为 `struct`

C 语言提供的 `struct` 数据类型的构造函数 (constructor) 与 C++ 和 Java 的对象最为接近。它允许程序员在一个数据结构中保存关于某个实体的信息, 并用名字来引用这些信息。

例如, 一个图形程序可能要用结构来表示一个长方形:

```
struct rect {
    long llx;           /* X coordinate of lower-left corner */
    long lly;           /* Y coordinate of lower-left corner */
    unsigned long width; /* Width (in pixels) */
    unsigned long height; /* Height (in pixels) */
    unsigned color;      /* Coding of color */
};
```

可以声明一个 `struct rect` 类型的变量 `r`, 并将它的字段值设置如下:

```
struct rect r;
r.llx = r.lly = 0;
r.color = 0xFF00FF;
r.width = 10;
r.height = 20;
```

这里表达式 `r.llx` 就会选择结构 `r` 的 `llx` 字段。

另外, 我们可以在一条语句中既声明变量又初始化它的字段: